

# OMNI CYBERSECURITY CHATBOT

## Project Part 2 Report



**Student Name: Omphile H Chaane**

**Student Number: ST10472730**

**Module: PROG6221 POE**

**Project: Cybersecurity Awareness Chatbot**

## **1. Introduction**

This project focuses on the development of a cybersecurity awareness chatbot named OMNI. The chatbot was designed using WPF and C# in Visual Studio. The main objective of the project was to create an interactive graphical user interface that helps users learn about cybersecurity topics such as phishing, malware, passwords, VPNs, and encryption.

## **2. GUI Design and Implementation**

The chatbot was designed using Windows Presentation Foundation (WPF). The graphical user interface contains a header section, chatbot title, logo, chat display area, user input textbox, and a send button. The interface uses a dark cybersecurity theme with blue highlights to create a modern and professional appearance.

The logo was placed in the top-left corner of the application window using an Image control in XAML. The GUI was structured using Grid layouts, Borders, StackPanels, and RichTextBox controls.

## **3. Keyword Recognition**

The chatbot recognises cybersecurity-related keywords entered by the user. A Dictionary collection was used to store keywords and their related responses. Examples of keywords include phishing, malware, password, VPN, and encryption.

When a user types a message, the chatbot scans the input using conditional checks and dictionary searches. If a keyword is detected, the chatbot responds with relevant cybersecurity advice.

## **4. Random Responses**

Random responses were implemented to make the chatbot more interactive and realistic. Instead of returning the same answer every time, the chatbot selects a random response from an array of predefined messages.

The Random class in C# was used together with arrays stored inside dictionaries. This allows the chatbot to provide varied and engaging conversations.

## **5. Conversation Flow**

The chatbot maintains a natural conversation flow by continuously accepting user input without restarting the application. Users can continue asking follow-up questions about cybersecurity topics.

The RichTextBox control was used to display both user messages and chatbot responses in a readable chat format.

## **6. Memory and Recall**

Memory functionality was implemented using dictionaries. The chatbot stores important information such as the last discussed topic. This allows OMNI to reference previous conversations and personalise responses.

For example, if the user previously discussed phishing, the chatbot can remind the user about that topic later in the conversation.

## **7. Sentiment Detection**

Simple sentiment detection was implemented to improve user interaction. The chatbot checks for words such as sad, angry, upset, happy, and good.

If a negative sentiment is detected, OMNI responds with supportive messages. If positive words are detected, the chatbot replies with encouraging responses.

## **8. Error Handling and Edge Cases**

Error handling was implemented using try-catch blocks to prevent the application from crashing. The chatbot also checks for empty or invalid user input.

If the chatbot does not recognise a keyword, it provides a default response asking the user to ask another cybersecurity-related question.

## **9. Code Optimization**

The project uses dictionaries, arrays, methods, and classes to keep the code organised and efficient. Functions such as AddBotMessage, AddUserMessage, and DetectSentiment were separated into reusable methods.

This structure improves readability, maintainability, and allows future expansion of the project.

## **10. Conclusion**

In conclusion, the OMNI Cybersecurity Chatbot successfully meets the project requirements for POE Part 2. The chatbot includes a graphical user interface, keyword recognition, random responses, memory functionality, sentiment detection, error handling, and code optimisation. The project demonstrates practical use of C#, WPF, and object-oriented programming concepts.